Reflexión breve: tres errores encontrados, como los solucionaste y que aprendiste

Ruben Antonio Herrera Rodriguez:

Error 1 Decimal sin importar: Al ejecutar specifications.py salía un error porque Decimal no estaba importado. Lo solucioné agregando from decimal import Decimal al inicio del archivo. Aprendí que los valores monetarios se manejan con Decimal (no float) para evitar errores de redondeo, y que hay que importarlo explícitamente.

Error 2 Mensaje de rechazo mejorado en confirm(): El método rechazaba la confirmación con un solo mensaje que no se distinguía si el motivo era un cliente inactivo o un pedido sin artículos, y eso chocaba con el escenario E03 (O003, cliente inactivo). Lo solucioné separando las validaciones en domain.py: primero revisa si el cliente está activo ("El cliente esta inactivo") y luego si hay artículos ("El pedido requiere articulos"). Aprendí que los mensajes de error deben indicar la causa exacta, porque ayudan a depurar y a que la bitácora de escenarios sea clara.

Error 3: ModuleNotFoundError: No module named app. Al ejecutar el programa apareció este error porque lo corrí desde una carpeta equivocada. Lo solucioné ejecutando los comandos desde la carpeta raíz, donde está cargar_csv.py. Aprendí que importa desde dónde se ejecuta cada comando.

Blanca Jocelin Vázquez Reyna:

Tres errores que encontramos en el equipo y cómo los resolvimos:

1. Confundir los archivos y revolver el código: Al principio, por las prisas con el punto 12, terminé pegando el código de las pruebas dentro del archivo del proyecto (app/domain.py) y por eso todo tronaba con errores de importación y módulos no encontrados (ModuleNotFoundError). **¿Cómo lo arreglé?:** Usé un comando de Git (git restore) para dejar el archivo original limpio como estaba, y ya pasé las pruebas a su carpeta correspondiente (tests/test_domain.py), asegurándome de correr siempre los comandos desde la carpeta raíz del proyecto.

2. Querer cambiar el código original en lugar de ajustar la prueba (Atributos VIP): Las pruebas de los descuentos fallaban porque el sistema usaba is_vip y yo quería modificar a fuerzas la clase Order para que aceptara otra cosa, lo cual chocaba con las reglas del dominio. **¿Cómo lo arreglé?:** Me acordé de la indicación de no tocar el archivo domain.py. Así que mejor corregí la prueba usando el atributo real (is_vip) e importé una función que ya venía en el proyecto (discount_rate desde app.specifications) para validar las promos sin romper nada.

3. Que las pruebas de la página (API) se estorbaran entre sí: Las pruebas de los endpoints web tronaban porque todas intentaban usar los mismos números de pedido al mismo tiempo y se hacían bolas con la memoria. **¿Cómo lo arreglé?:** Separé las pruebas en test_api.py para que cada una fuera totalmente independiente. Le asigné un pedido diferente del archivo CSV a cada caso (por ejemplo, el O013 para la doble confirmación y el O012 para el envío) y ordené el código paso a paso: armar los datos, hacer la acción y revisar que saliera bien.

¿Qué me dejó de aprendizaje? Aprendí la regla de oro en el desarrollo: si una prueba falla, primero hay que revisar que la prueba esté bien hecha antes de querer moverle al código fuente que ya funciona. También entendí la importancia de estructurar las pruebas paso a paso usando el patrón de Preparar-Ejecutar-Comprobar (Arrange-Act-Assert), a cuidar desde qué carpeta ejecutamos los comandos en la terminal, y a ver que en las pruebas web es súper necesario que cada una vaya por su lado para no afectar los resultados de los demás.

Aaron Sebastian Torres Cabriales

1. **Falta de importaciones en las especificaciones:** Al momento de correr el archivo specifications.py, el programa se detuvo por completo porque faltaba importar el tipo de dato Decimal. **¿Cómo lo arreglé?:** Agregué la línea from decimal import Decimal al principio del archivo. Aprendí que en proyectos financieros o de pedidos siempre hay que usar Decimal en lugar de float para evitar errores raros en los centavos y los cálculos de dinero.

2. **Mensajes de rechazo confusos en el dominio:** El método para confirmar pedidos rechazaba las operaciones con un mensaje muy general, lo que causaba que chocara con las pruebas del escenario E03 cuando un cliente estaba inactivo. **¿Cómo lo arreglé?:** Modifiqué el código en domain.py para separar las validaciones y dar mensajes más claros: primero revisa si el cliente está inactivo y luego si el pedido no tiene artículos. Aprendí que ser específicos con los mensajes de error nos ahorra dolores de cabeza al momento de depurar.

3. **Problemas al ejecutar por la ubicación en la terminal:** Al intentar correr las pruebas, la consola me marcaba un error de que no encontraba el módulo app (ModuleNotFoundError). **¿Cómo lo arreglé?:** Me di cuenta de que estaba parado en una carpeta equivocada, así que me cambié a la carpeta raíz del proyecto (donde está el archivo cargar_csv.py) y volví a lanzar el comando. Aprendí que la ruta desde donde ejecutas la terminal es clave para que los scripts encuentren los archivos.

¿Qué me dejó de aprendizaje? Aprendí que cuidar los detalles pequeños, como importar bien los tipos de datos numéricos y estructurar los mensajes de error del sistema, hace que todo fluya mejor